

计算机体系结构

08. 流水线基础

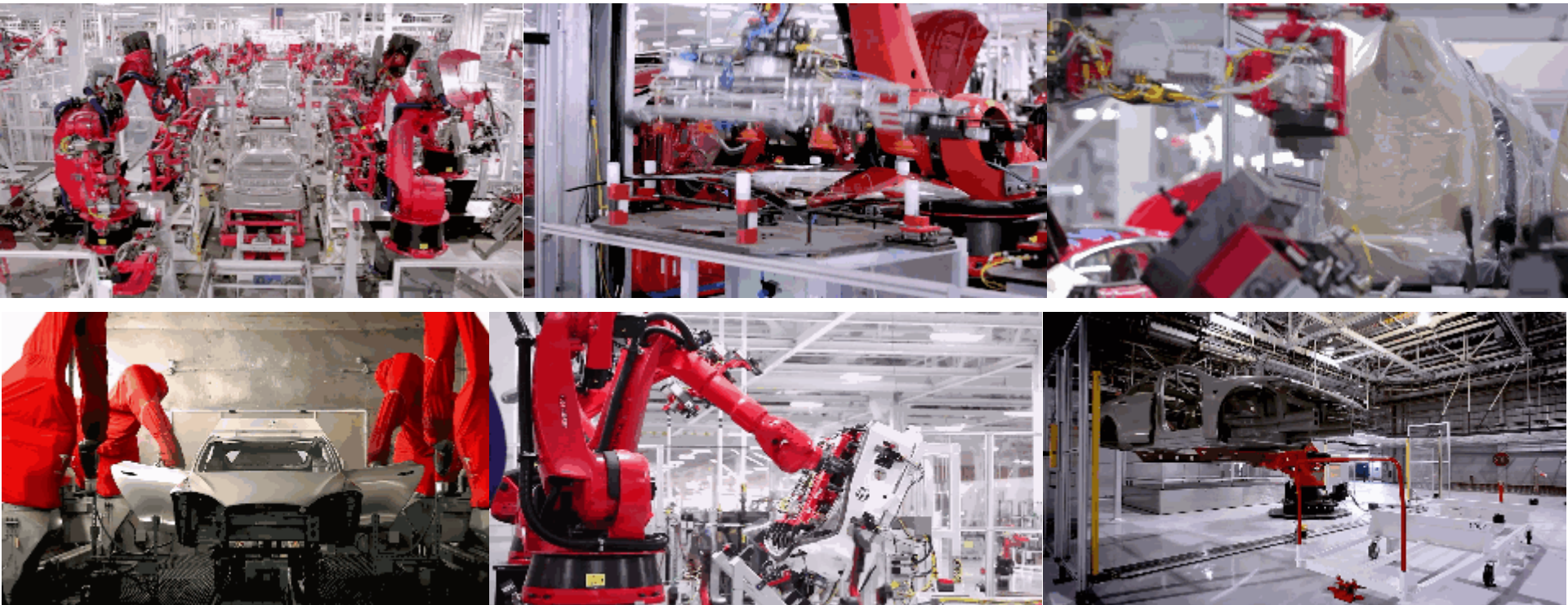
李建华

计算机与信息学院

The contents of the slides are adapted from CA course of wisc, princeton, mit, berkeley, edinburgh, and eth.
The uses of the slides of this course are for educational purposes only and should be used only in conjunction with the textbook. Derivatives of the slides must acknowledge the copyright notices of this and the originals.

流水线回顾

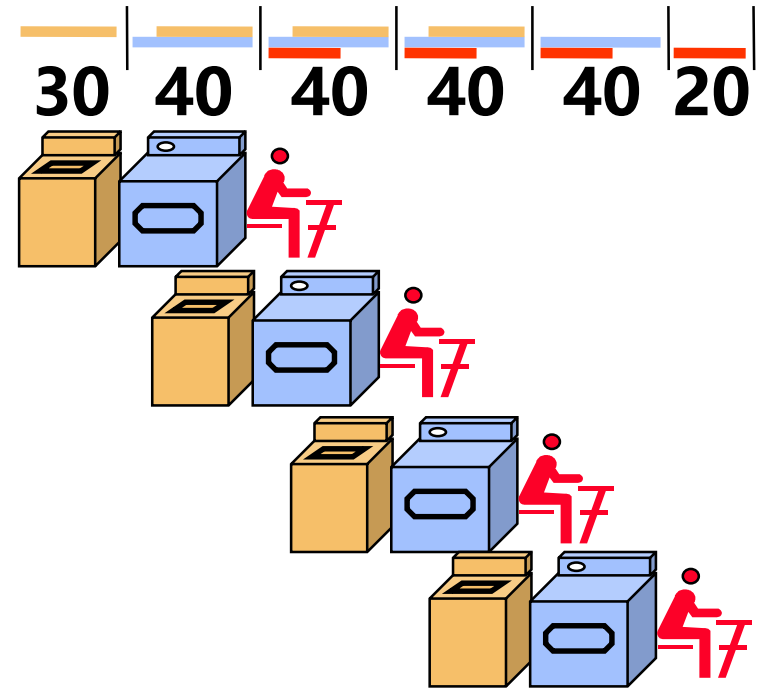
- 流水线的主要特性有哪些？
- 流水线能获得的加速比是多少？



<http://www.ilinki.net/news/detail/26189>

关于流水线的共性结论

- 流水线不能缩短单个任务的响应时间，但可以提高吞吐率；
- 流水线速度受限于最慢流水段的速度；
- 流水线中多个任务是并行处理的，且处于不同的处理阶段；
- 理想的加速比 $\approx$ 流水段的数量
 - 流水段的速度不均衡
 - 流水线“填充”和“排空”时间
 - 其它干扰流水线运行的因素



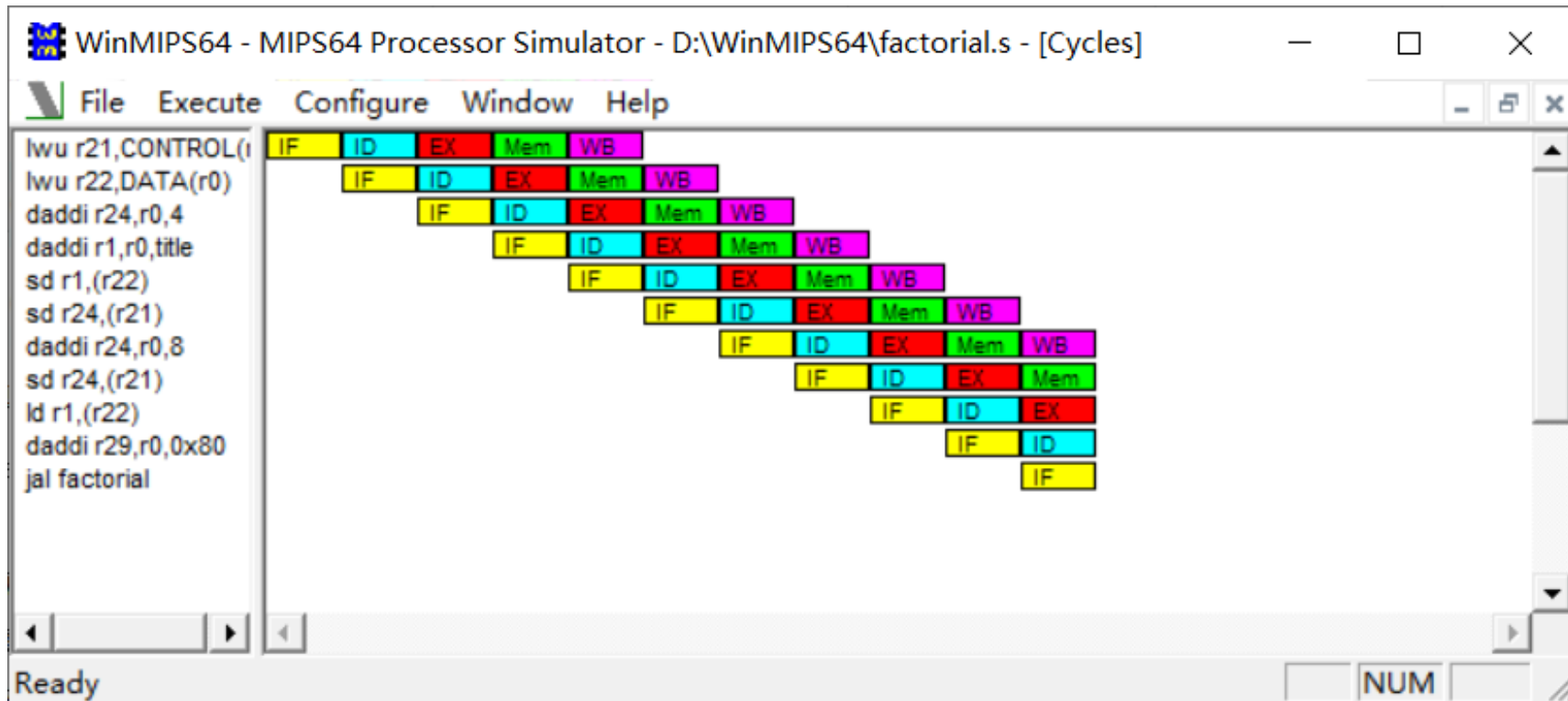
本讲提纲

- 指令流水线
- MIPS的流水处理
- 流水线面临的主要问题
- 相关及其分析

什么是指令流水线?

- 指令流水线:
 - 将多条指令的处理进行流水操作，重叠执行。
- 实现方法:
 - 将指令的处理划分为不同的阶段 (Stage)
 - 确保每个阶段**有独立的硬件资源**专门处理工作
 - 同一时刻，每个阶段处理不同的指令：
 - **程序顺序**中连续的指令，在流水线中连续的阶段中处理。
- 指令流水线的好处:
 - 可提升指令处理的吞吐率 (throughput)

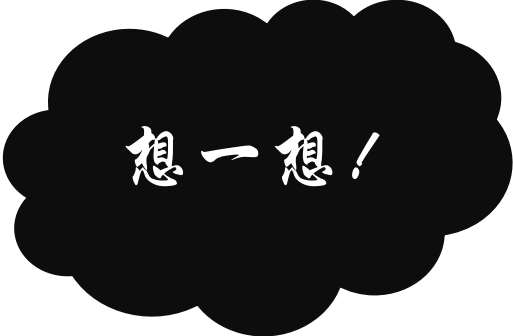
指令的流水线处理



- 对于MIPS来说，指令的处理分为5个阶段：
 - IF, ID, EXE, MEM, WB

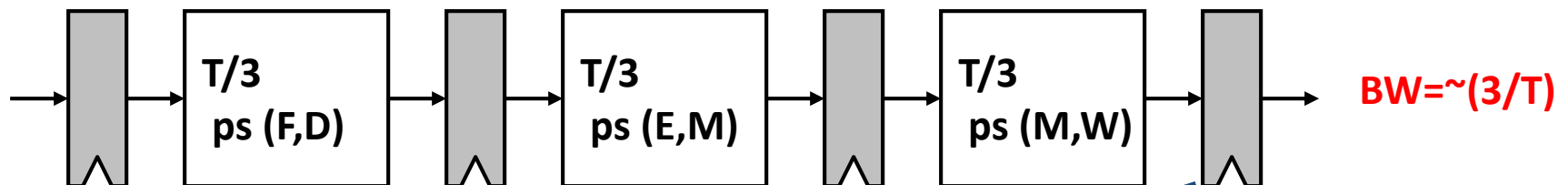
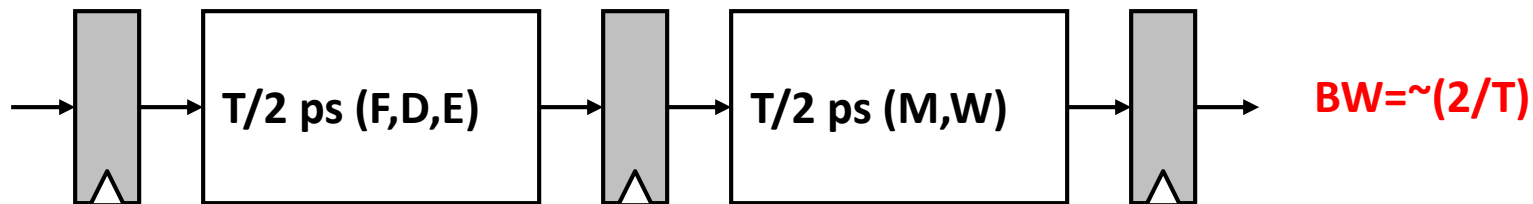
理想的指令流水线

- 理想的Pipeline，有何特性？
- 处理的是**相同的指令**
 - 对大量数据进行相同的操作，比如累加；
- 处理的**指令之间是相互独立的**
 - 流水线上处理的指令之间，没有依赖关系；
- 处理的**指令可以均匀划分成一些子操作**
 - 指令的处理可以划分为所需时间相等的子操作（流水段）



想一想！

理想的流水线模型

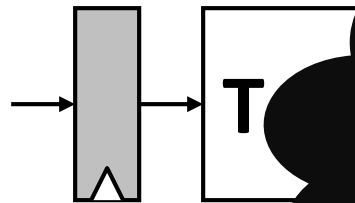


为什么需要这些锁存器 (latch) ?

现实的流水线: 吞吐率

- 非流水版本, 延迟为: T

$BW = 1/(T+S)$, S 表示锁存器的延迟



流水线的级数越多性能越好吗?

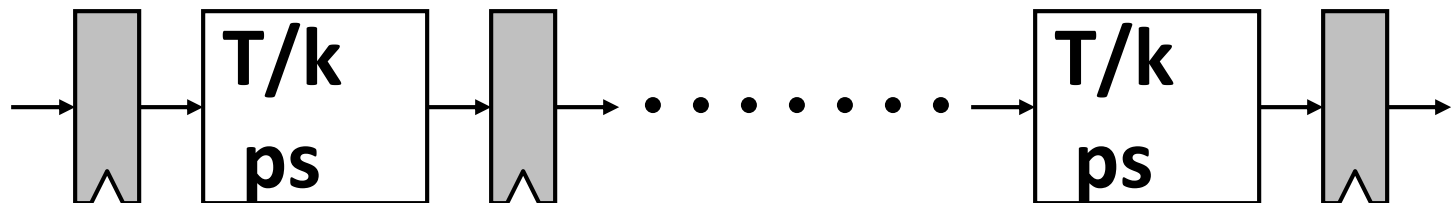
Think about it!

- K阶流水线版本

$$BW_{k\text{-stage}} = 1 / (T/k + S)$$

$$BW_{\max} = 1 / (1 \text{ gate delay} + S)$$

寄存器的延迟会限制吞吐率



现实的流水线: 开销

- 非流水版本的组合逻辑开销为: G

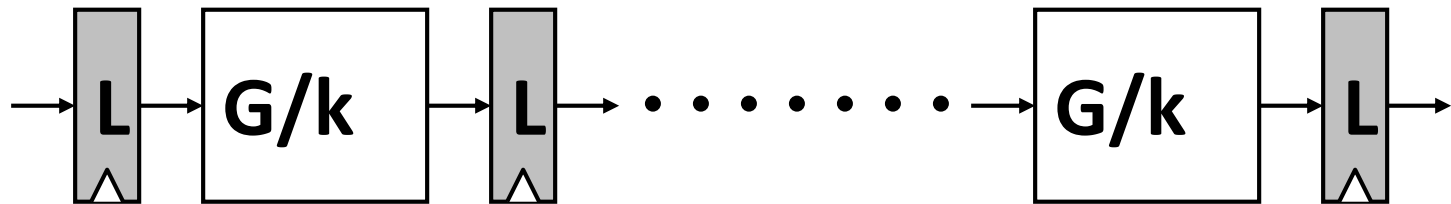
$\text{Cost} = G + L$, L 表示锁存器的开销



- K阶流水线版本

$$\text{Cost}_{k\text{-stage}} = G + Lk$$

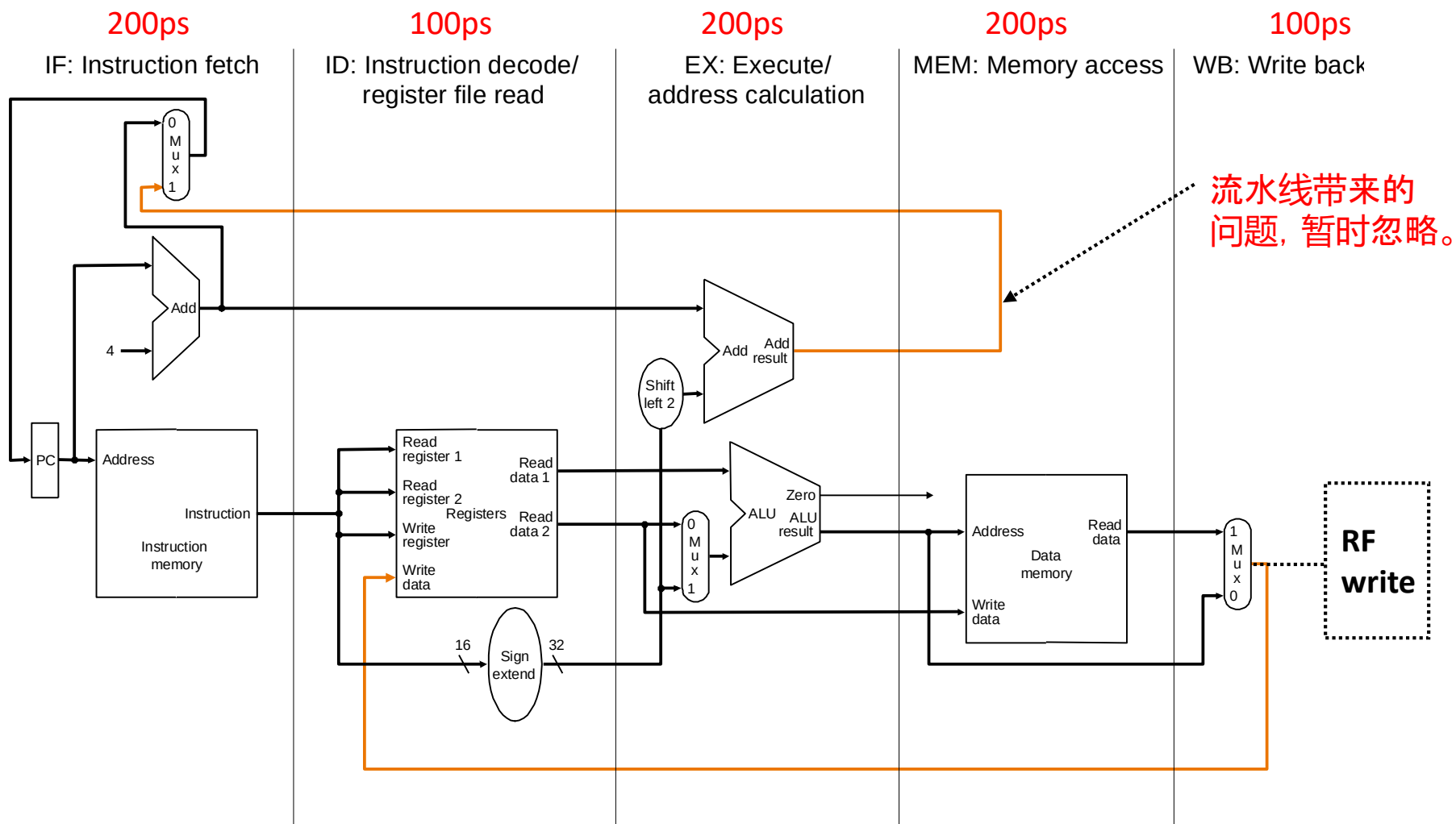
锁存器增加了硬件开销



本讲提纲

- 指令流水线简介
- MIPS的流水处理
- 流水线面临的主要问题
- 相关及其分析

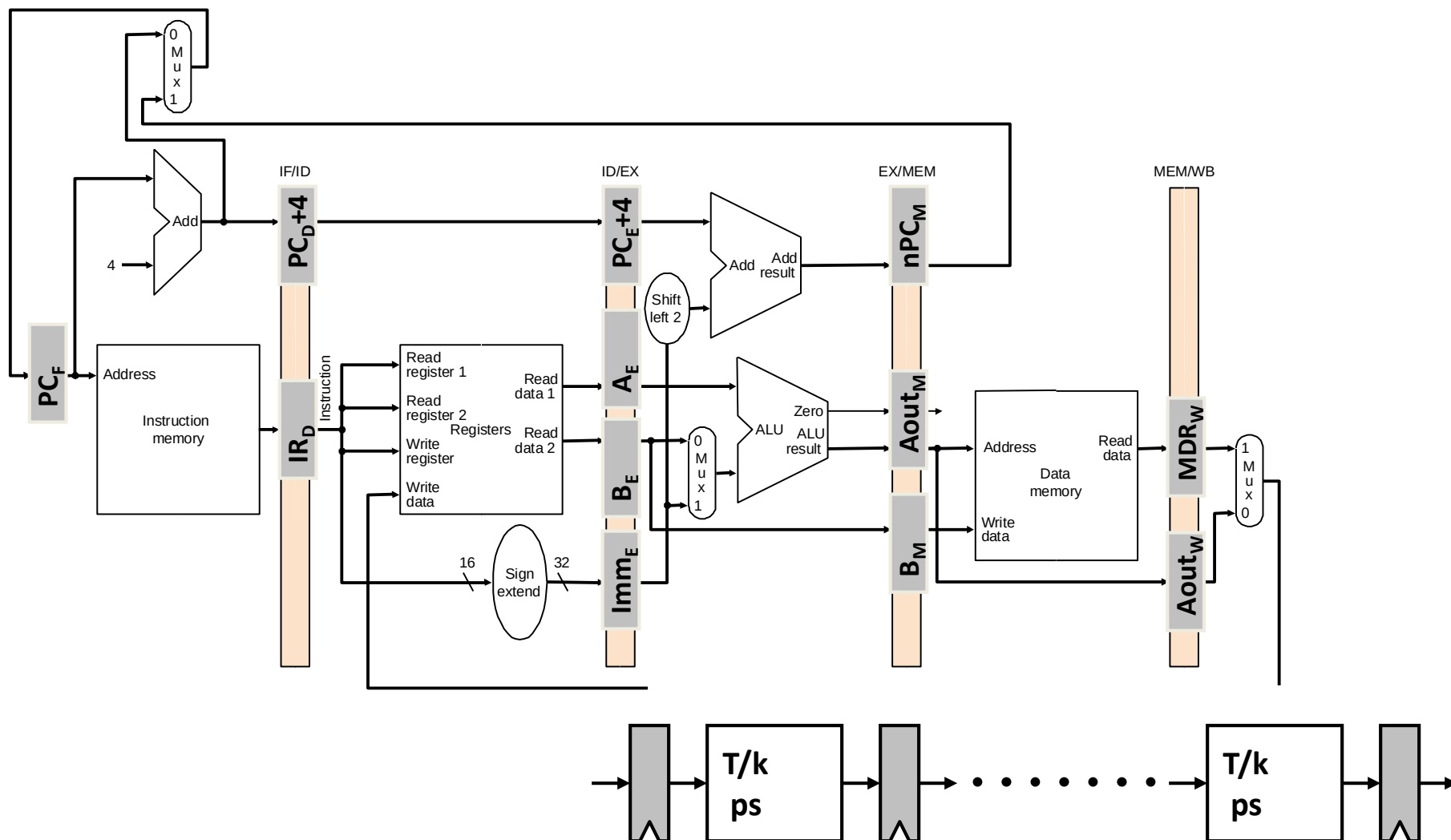
将指令处理分割为多个周期



- 为什么不是4个或者6个阶段?
- 为什么不是其它的边界划分?

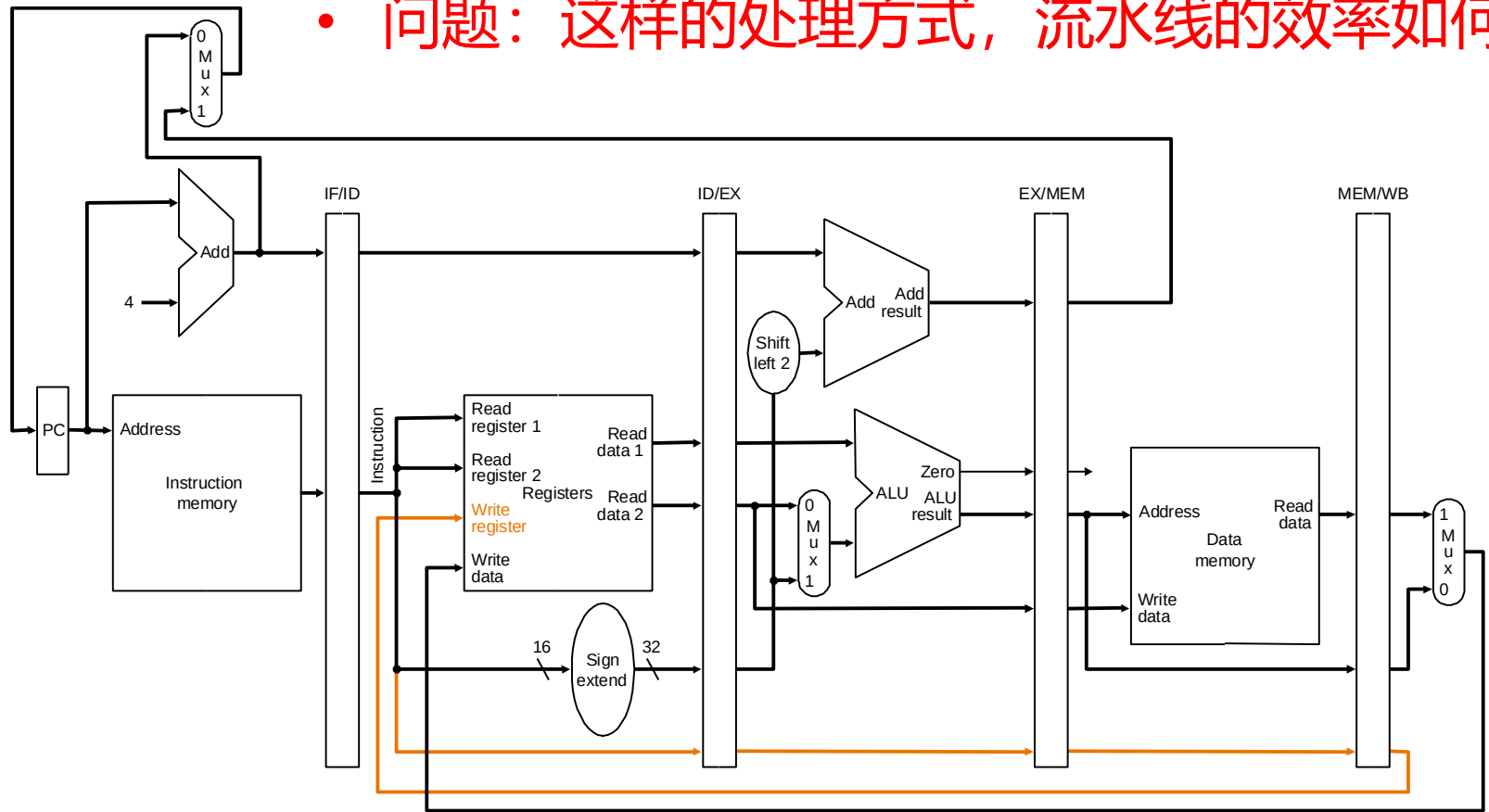
流水线寄存器

没有任何一个硬件单元被超过一个流水段同时使用!

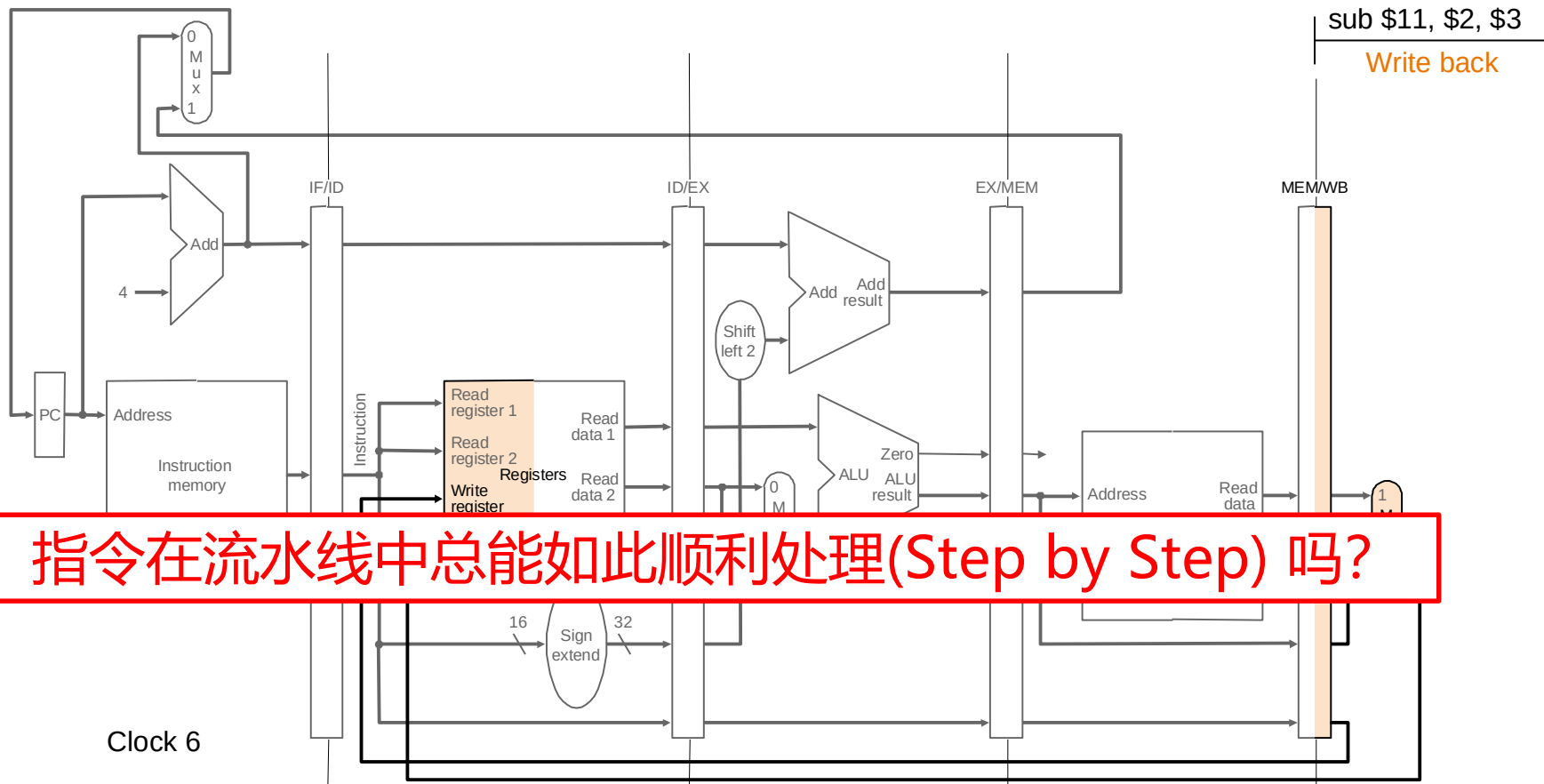


流水线操作示例

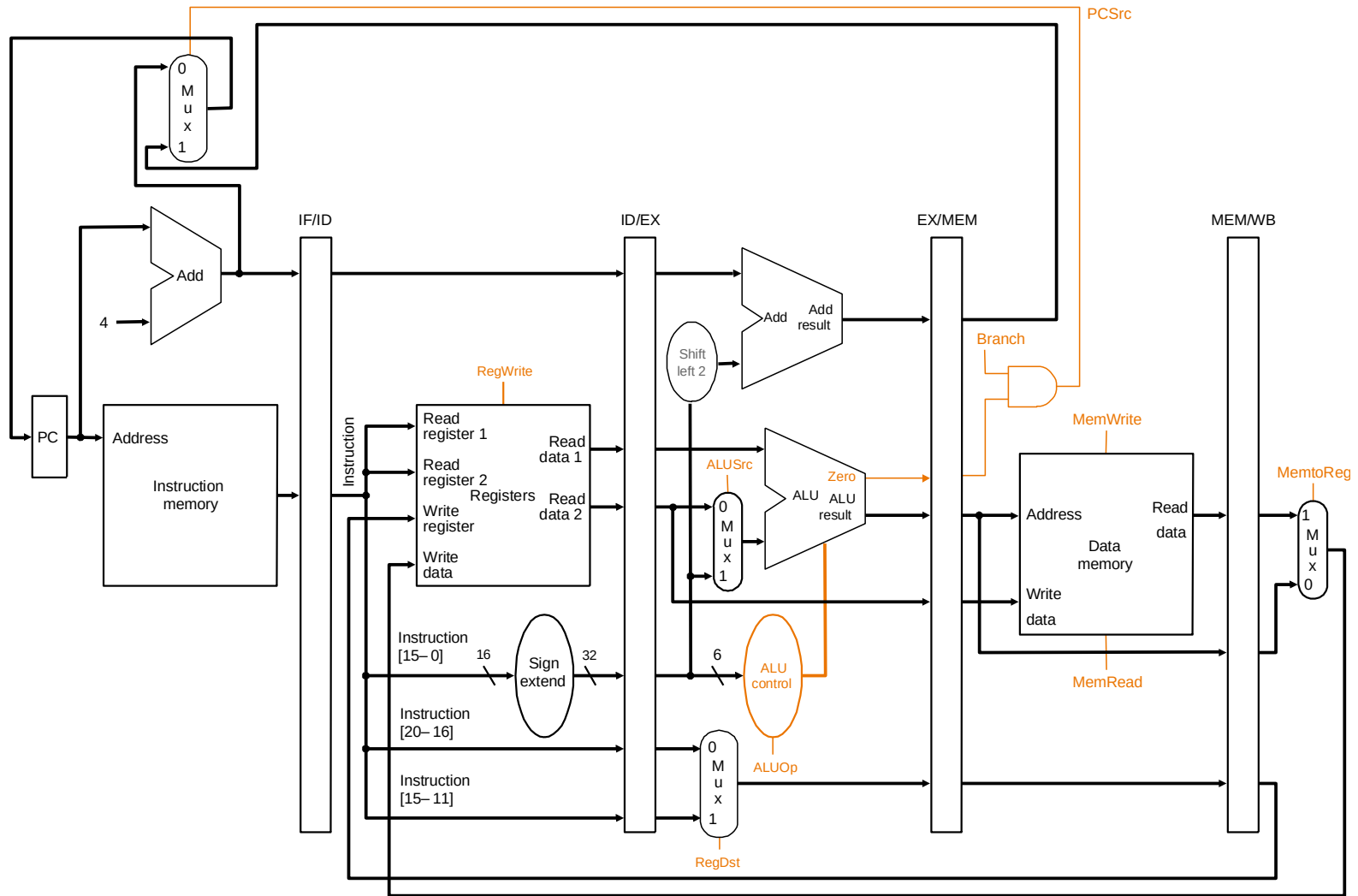
- 所有指令按照相同的数据通路通过各流水段
- 问题：这样的处理方式，流水线的效率如何？



流水线操作示例



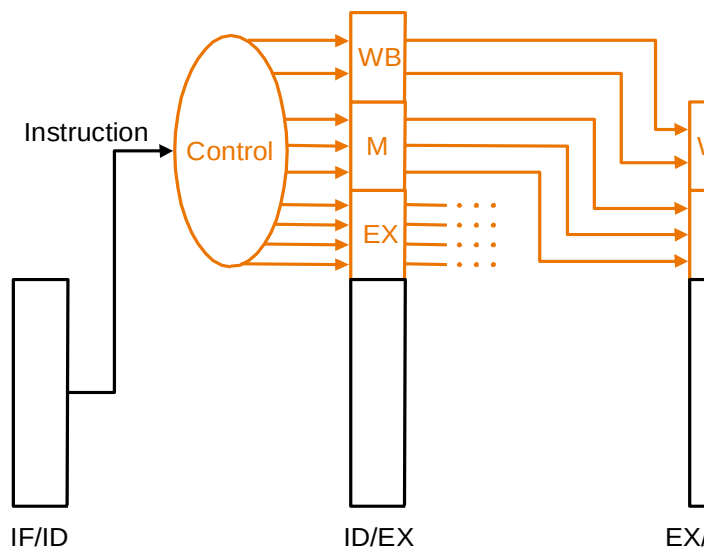
流水线的控制



- 与单周期设计相比，所需要的控制信号完全相同！

流水线中控制信号的处理

- 对于一条给定的指令来说
 - 与单周期设计的控制信号相同
 - 但是，这些控制信号在不同的流水段（不同的时间）使用
- ⇒ 方法1: 像单周期一样一次译码生成**大部分**控制信号



⇒ 方法2: 将指令中与控制信号的生成有关的部分的工作，来产生控制信号。

- 哪一种方法更好呢？

```
always@(negedge clk) begin
    if(hit) begin

        ReadData1Out <= ReadData1;
        ReadData2Out <= ReadData2;
        SignExtendImmediateOut <= SignExtendImmediate;

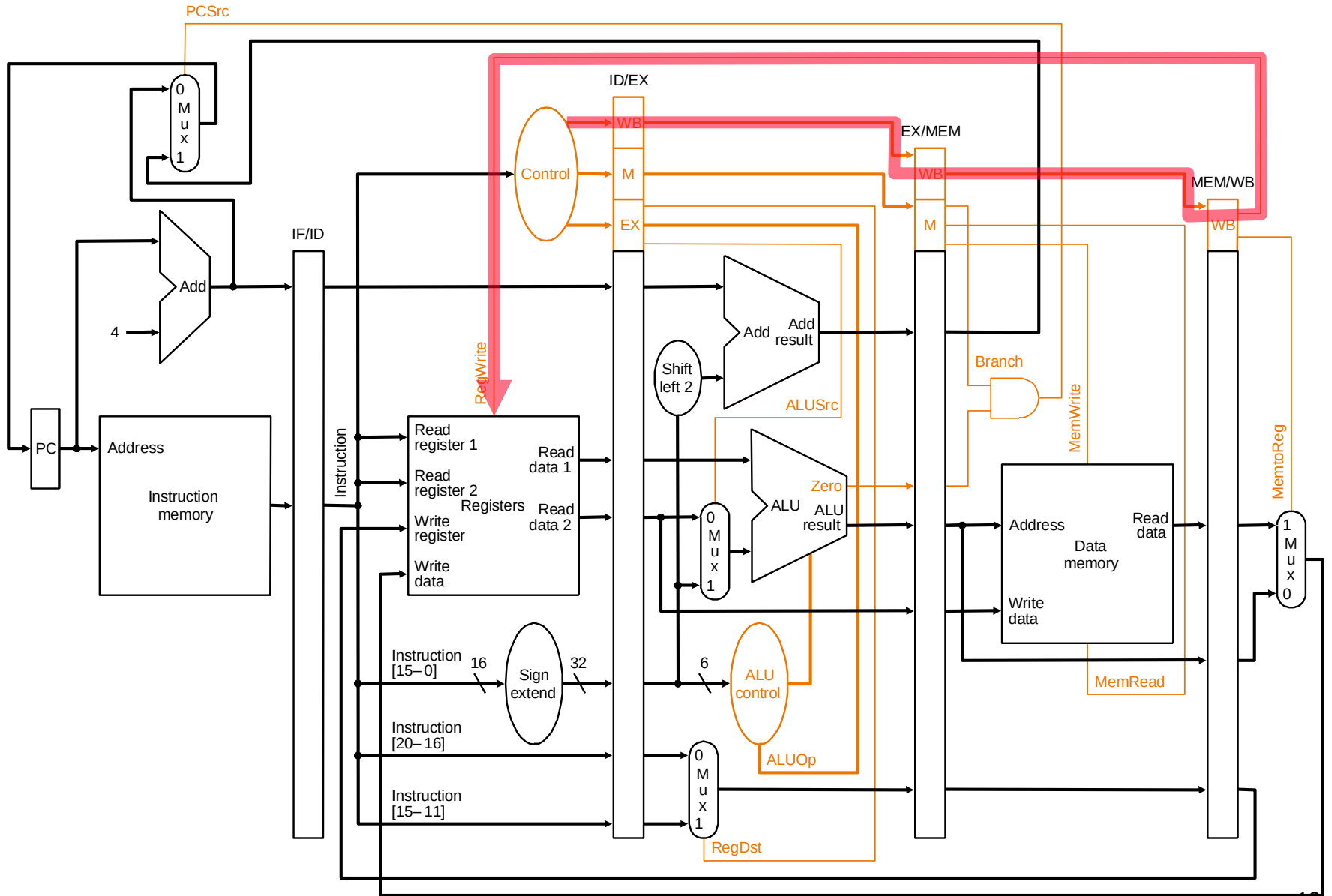
        functOut <= funct;
        ALUSrcOut <= ALUSrc;
        RegDstOut <= RegDst;

        MemtoRegOut <= MemtoReg;
        RegWriteOut <= RegWrite;
        MemReadOut <= MemRead;
        MemWriteOut <= MemWrite;

        nextPCOut <= nextPC;
        BranchOut <= Branch;
        ALUOpOut <= ALUOp;
        rdOut <= rd;
        rtOut <= rt;

    end
end
```

流水化的控制信号



本讲提纲

- 指令流水线简介
- MIPS的流水处理
- 流水线面临的主要问题
- 相关及其分析

指令流水线: 理想 vs 现实

- 相同的操作... 并不是!

- ⇒ 各不相同的指令 → 并不是总需要经过所有流水段

- 流水线的的设计强制所有的指令沿着相同的流水段进行处理

- 可能会导致外部碎片 (对某些指令的处理, 某些流水段处于空闲状态)

- 均匀的流水段... 并不是!

- ⇒ 不同的流水段 → 所需要的延迟并不是完全相同的

- 流水线的的设计中, 每个流水段需要相同的时钟来进行控制

- 可能会导致内部碎片 (有些流水段处理很快, 但是最终所反映出来的处理时间是相同的。)

- 相互独立的操作... 并不是!

- ⇒ 指令之间并不是相互独立的

- 需要检测并解决指令之间的依赖关系, 来确保流水线产生正确的结果。

- 可能会导致流水线暂停 (流水线并不是一直是流动的)

设计流水线面临的问题

- 首先，如何均衡各个流水段的工作量？
 - 划分为多少个流水段，每个流水段做什么？
 - 如何设计才能均衡各段的延迟？ **【在此我们不讨论】**
- 其次，在遇到破坏流水线正常处理的事件时，如何确保流水线正确、满负荷运行？
 - 比如：如何处理异常和中断？
- 最后，如何优化流水线的吞吐率？
 - 最小化流水线暂停

引起流水线停顿的主要原因

- 什么是停顿 (stall) : 导致流水线必须停止向后流动的事件发生, 从而必须停止流水处理。
- 造成流水线停顿的主要原因:
 - ① 资源冲突 : 多条指令同时使用同一份资源
 - ② 相关 (指令之间) 造成流水线冒险
 - 数据相关
 - 控制相关
 - ③ 长延迟 (多周期) 操作破坏正常流水节奏
 - 如浮点乘除操作

本讲提纲

- 指令流水线简介
- MIPS的流水处理
- 流水线面临的主要问题
- 相关及其分析

相关(dependence)及其类型

- 在某些教材上叫做: “hazard” 冒险, 但不严谨!
- 相关限定了指令执行时的顺序要求, 限制了一些优化策略 (例如: 指令调度) 的应用。
 - 冒险和相关是二个独立的概念
- 相关的分类:
 - 数据相关 (Data dependence)
 - 控制相关 (Control dependence)
- 资源冲突有时候又叫 **“资源相关”**
 - 资源冲突和程序的语义没有本质的关系, 是系统资源配置造成的问题, 因此很少提 **资源相关**。

资源冲突的处理

- 当处于二个流水段中的指令需要相同的资源（而该资源只有一份）就会发生资源冲突；
- 解决方案1: 消除引起冲突的原因
 - **重复资源配置** 或者 增加资源处理的吞吐率
 - 例如，使用分离的指令和数据缓存
 - 再例如，使用多个端口的存储器
- 解决方案2: **检测**资源冲突，并**暂停**其中一个流水段中指令的处理。【调度机制】
 - 暂停哪个流水段中指令的处理？
 - 例如: 寄存器文件只有一个读端口和一个写端口，如何处理ADD指令的访问请求？

数据相关 (Data Dependence)

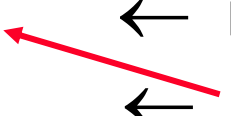
- 数据相关的类型
 - 真相关 (read after write, RAW), 又叫流相关;
 - 输出相关 (write after write, WAW);
 - 反相关 (write after read, WAR);
- 上述哪一种相关会造成流水线处理器的暂停?
 - 对上述所有相关我们需要确保程序的计算结果是正确的
【这是底线，结果正确是计算的目的。】
 - 真相关是必须一直要遵从的，因为构成了指令间基于值的依赖关系
 - 反相关和输出相关又叫名相关，其出现的原因，是由于体系结构中的硬件寄存器数量不足。
 - 它们相关于同一个寄存器名，而不是一个值。

都有可能!

数据相关示例

Flow dependence (真相关)

$r_3 \leftarrow r_1 \text{ op } r_2$ Read-after-Write
 $r_5 \leftarrow r_3 \text{ op } r_4$ (RAW)



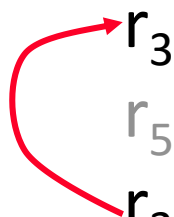
Anti dependence (反相关)

$r_3 \leftarrow r_1 \text{ op } r_2$ Write-after-Read
 $r_1 \leftarrow r_4 \text{ op } r_5$ (WAR)

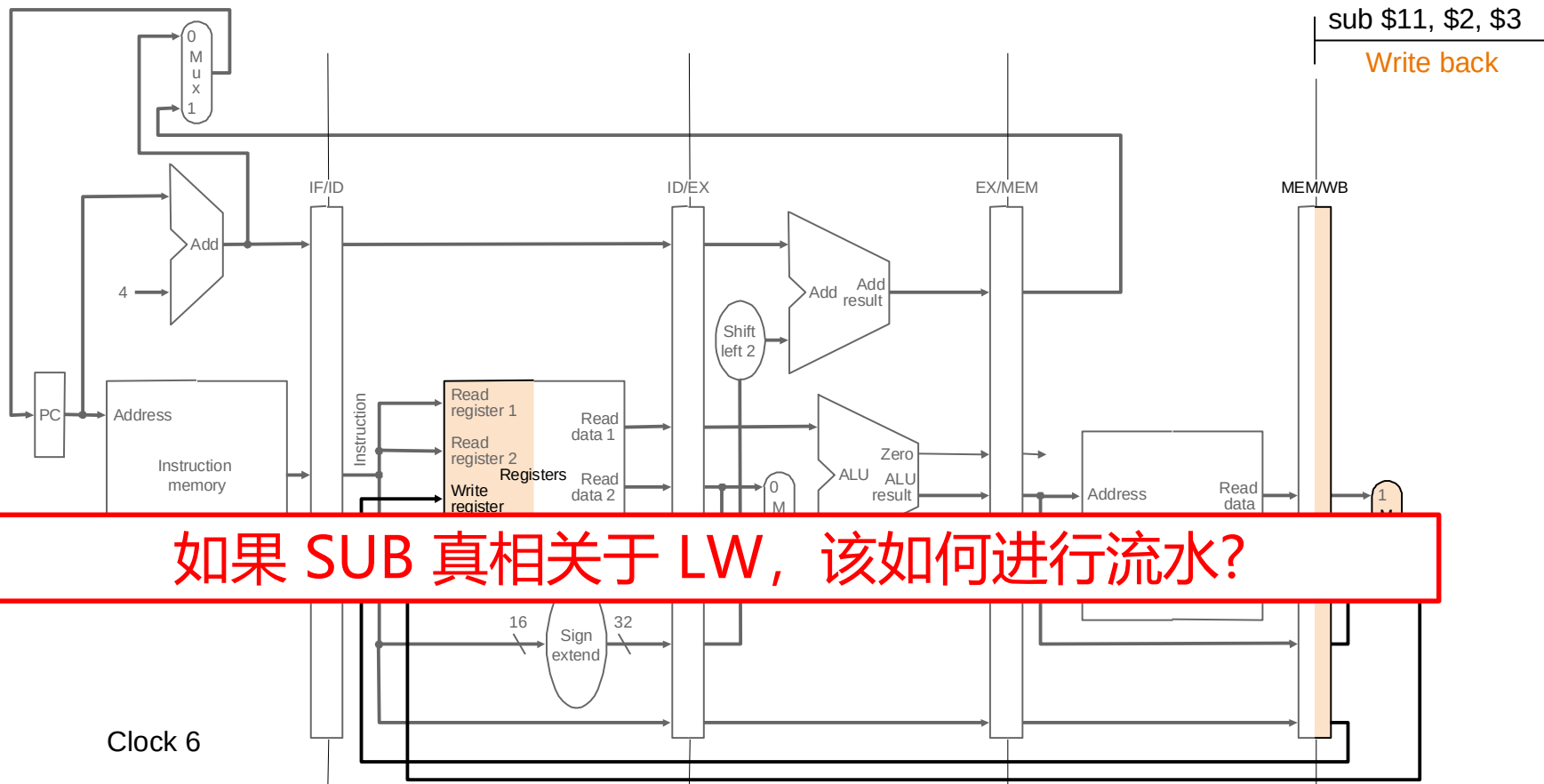


Output-dependence (输出相关)

$r_3 \leftarrow r_1 \text{ op } r_2$ Write-after-Write
 $r_5 \leftarrow r_3 \text{ op } r_4$ (WAW)
 $r_3 \leftarrow r_6 \text{ op } r_7$



指令流水线操作示例



如何处理数据相关?

- 反相关和输出相关相对容易处理
 - 在某一个阶段按照**程序顺序**来写目标寄存器即可
- 流相关对程序执行结果比较关键，难以处理；
- 比较典型的处理流相关的方法：
 - **检测**并等待，直到寄存器中相关的值已经就绪
 - **检测**并前推，将值前推给需要该值的指令
 - **检测**并消除，在软件层面消除真相关
 - 不需要硬件来检测相关，通常由编译器处理
 - **预测**消费者指令需要的值，并推测地执行和验证
 - [Value prediction](#)

相关检测方法 (1)

- 记分板检测
 - 为寄存器文件中每个寄存器配置**有效位 (valid bit)**
 - 写某个寄存器的指令要**重置 (reset)** 相应的有效位
 - 一个处于译码阶段的指令检查：其所需的**源和目的寄存器**的有效位是否为valid?
 - 是: 不需要暂停... 无数据相关，重置目的寄存器的有效位，继续流水处理。
 - 否: 有数据相关，暂停该指令的处理。
- 记分板的优点：
 - 实现简单，每个寄存器1比特开销
- 记分板的缺点：
 - **输出和反相关**也要暂停，不利于流水线性能。

相关检测方法 (2)

- 基于组合逻辑的数据相关检测
 - 用专门的**检测逻辑**来检测**当前处于后续流水段中的指令的目标寄存器**是否是处于**译码阶段的指令的源寄存器**?
 - 是，暂停译码段中指令的处理；
 - 否，没必要暂定流水线... 没有真相关
- 组合逻辑检测法的优点：
 - 输出相关时，没必要暂停流水线。
- 组合逻辑检测法的缺点：
 - 检测电路比记分板算法中的逻辑开销大
 - 对于深度流水线和超标量处理器来说，开销更大。

真相关的处理

- 检测到真相关之后，如果想保障流水线的性能该如何处理？
 - 暂停流水线是好的方法吗？
- **关于真相关和指令间通信的观察：**
 - 二条指令存在真相关**最早何时**被检测到？
 - 真相关的指令在流水线中**最迟在何时**进行信息传递？
 - 上述2个事件是否**在同一个时刻**发生？
- 处理方案1: 立刻停止后续指令的处理（stall）
- 处理方案2: 仅在必要的时候暂定后续指令的处理
 - 数据前推/旁路（Forwarding/Bypass）

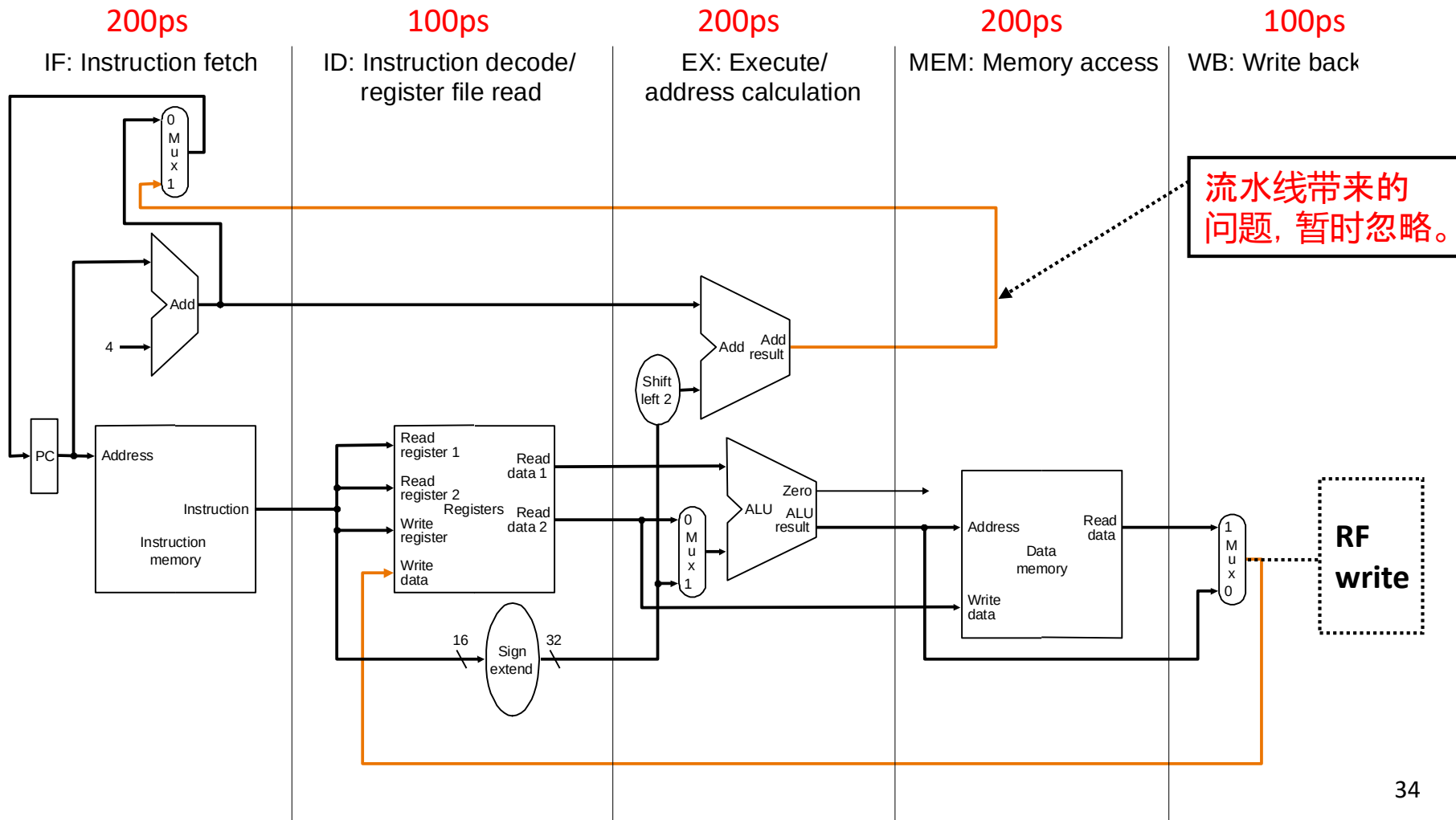
数据前推的原理分析

- **问题:** 一条消费者指令(**真相**关于生产者指令) 不得不在其**译码段等待**生产者指令将其计算结果写到目标寄存器中
- **目标:** 为了保障流水线的性能, 非万不得已不暂停流水线。
- **观察:** 消费者指令所需要的值也可以从流水线的**数据通路**中获取 (没有必要一定要在译码段从寄存器文件中读取)
- **想法:** 添加额外的相关检测逻辑, 以及数据前推所需的路径 (总线), 来将生产者指令产生的值 **直接传输** 给消费者指令。
- **好处:** 消费者指令可以一直沿着流水线处理, 直到到达产生值的那个流水阶段 → **较少的流水线暂停**
- 下一节, 我们会详细介绍数据前推。

控制相关

- 控制相关

- 数据相关于 PC (指令指针)



控制相关概要分析

- **问题:** 对于流水线处理器来说，下一个周期**取指**所依赖的PC是什么？
- **答案:** 下一条指令的地址！
 - 那么，所有的指令 **均控制相关于** 它前面的指令。为什么？
- **如果当前取到的指令是非控制流指令：**
 - 可以立刻确定下一条指令的地址 【例如：PC+4】
 - 只需要知道指令的宽度，很容易可以确定下一条指令的地址。
- **如果当前取到的指令是控制流指令：**
 - 如何立刻确定下一个PC呢？ 【貌似很困难】
- **挑战：** 实际上，我们如何/何时才能知道当前取到的指令是否是控制流指令呢？ **请思考这个问题！**

Next Topic

C.1	Introduction	C-2
C.2	The Major Hurdle of Pipelining—Pipeline Hazards	C-11
C.3	How Is Pipelining Implemented?	C-30
C.4	What Makes Pipelining Hard to Implement?	C-43
C.5	Extending the MIPS Pipeline to Handle Multicycle Operations	C-51
C.6	Putting It All Together: The MIPS R4000 Pipeline	C-61
C.7	Crosscutting Issues	C-70
C.8	Fallacies and Pitfalls	C-80